Relatório – Testes Automatizados do Truco Gaudério

Disciplina de Qualidade de Software 2026/02 · Renato e Lucas · Nov 11, 2026 · @Renato

Introdução

Este relatório documenta, etapa por etapa, os testes automatizados com **PyTest** que aplicamos ao jogo *Truco Gaudério com CBR*, escrito em Python. O jogo coloca um humano contra um bot que decide suas jogadas por raciocínio baseado em casos (CBR), usando os 100 vizinhos mais próximos numa base de partidas.

Decisão de abordagem: testamos o sistema **como ele foi entregue, sem corrigir o código do jogo**. Quando um teste revela um defeito, ele é marcado como `xfail` (falha esperada), com o motivo descrito. Assim a suíte continua verde e serve como registro dos defeitos encontrados.

Item	Detalhe
Sistema sob teste	Pacote `truco/`, com 12 módulos: carta, baralho, jogador, bot, jogo, truco, envido, flor, interface, dados, cbr e __main__
Ferramenta	PyTest, com pytest-cov para medir a cobertura
Base de casos da IA	`dbtrucoimitacao_maos.csv`, com cerca de 14 MB
Entregas	Documento de requisitos e casos de teste, `RenatoLucastestesoftware.zip` e `RenatoLucastestesoftware.mp4` (de 5 a 15 min)
Prazo	As apresentações começam em 16/11/2026 e a entrega final é em 23/11/2026, às 14h30

Tipos de teste exigidos no enunciado

Tipo de teste	Obrigatório?
Validação de requisitos e cenários de uso	Sim
Chamadas de função com parâmetros variados	Sim
Entradas de dados do usuário	Sim

Tipo de teste	Obrigatório?
Retornos de função	Sim
IA implementada no jogo	Sim
Exceções descritas no código	Não
Mensagens de erro	Não
Início e fim de laços	Não
Combinações de `if`s encadeados	Não

Etapas do trabalho

1. Preparação do ambiente de testes
2. Requisitos e casos de teste
3. Testes unitários por módulo
4. Testes da IA (CBR)
5. Testes de entradas do usuário e cenários ponta a ponta
6. Execução, cobertura e relatório de defeitos
7. Empacotamento e roteiro do vídeo

Etapa 1 – Preparação do ambiente de testes

Resultado: o ambiente PyTest está pronto. Os 22 testes de fumaça rodam em cerca de 4 segundos: 21 passaram e 1 falhou de propósito, comprovando o defeito D01. A cobertura inicial é de 38% do pacote `truco`.

Estrutura criada no projeto

```
trabalho_qualidade_software/
├── .venv/                    ambiente virtual Python (não vai para o Git)
├── configurar_ambiente.bat cria o .venv, instala as dependências e roda os
testes
├── pytest.ini                configuração do PyTest e dos marcadores
├── requirements-test.txt     pandas, scikit-learn, pytest e pytest-cov
├── .gitignore                ignora .venv, cache, cobertura e o jogadas.csv
├── truco/                    código do jogo (não alterado)
└── tests/
    ├── README.md             como rodar e como ler o resultado
    ├── conftest.py           fixtures compartilhadas
    └── test_00_ambiente.py testes de fumaça do ambiente
```

As dependências ficam isoladas num **ambiente virtual** `.venv` dentro do projeto. Assim as versões de pandas, scikit-learn e pytest usadas nos testes não se misturam com o Python do sistema, e qualquer integrante do grupo consegue recriar o mesmo ambiente.

Configuração (pytest.ini)

- `--strict-markers` faz o PyTest recusar marcadores digitados errado.
- `xfail_strict = true` serve de alarme: se um defeito marcado como `xfail` deixar de acontecer, o teste aparece como XPASS e falha.
- Os marcadores separam a suíte por categoria do enunciado. Assim dá para rodar só uma parte durante o vídeo.

Marcador	Uso	Exemplo de comando
`requisito(id)`	Liga o teste a um requisito ou caso de teste do documento	`pytest -m requisito`
`ia`	Testes da IA (CBR)	`pytest -m ia`
`entrada`	Entradas do usuário simuladas	`pytest -m entrada`
`cenario`	Partida completa executada pelo `__main__`	`pytest -m cenario`
`defeito(id)`	Comprova um defeito do jogo, sempre junto com `xfail`	`pytest -m defeito`
`lento`	Carrega a base de casos completa	`pytest -m "not lento"`
`sem_adaptador`	Roda como o usuário, sem o contorno do D01	—

Fixtures do conftest.py

As fixtures permitem testar o jogo sem mexer no código dele. Cada teste recebe só as peças de que precisa.

Fixture	O que faz	Para que tipo de teste
`nova_carta`, `nova_mao`	Criam cartas a partir de texto, como "1 de ESPADAS"	Todos
`baralho_fixo`	Baralho que entrega as cartas numa ordem conhecida	Requisitos e retornos
`jogador_com_mao`, `bot_com_mao`	Jogador e bot com mãos definidas pelo teste	Funções com parâmetros variados e `if`s
`entradas_usuario`	Simula o teclado (`input()`) e registra cada pergunta feita	Entradas do usuário e laços de validação
`cbr_falsa`	IA "dublê" com respostas controladas e registro das chamadas	Truco, Envido, Flor e Bot isolados
`cbr_real`	IA verdadeira com a base de 27.515 casos, carregada uma vez por execução	Testes da IA
`executar_partida`	Roda o `truco/__main__.py` inteiro com entradas, semente e mãos fixas	Cenários ponta a ponta
`interface`	Instância da classe Interface	Mensagens na tela (com `capsys`)

Todo teste roda numa pasta temporária. Isso evita que o arquivo `jogadas.csv`, que o jogo grava ao fim de cada mão, suje o projeto.

Se o jogo pedir mais entradas do que o teste forneceu, a simulação do teclado lança `EntradasEsgotadas`. Dessa forma um laço infinito de validação vira um erro que o teste consegue verificar, em vez de travar a suíte.

Defeito encontrado: D01 – o jogo não inicia

Ao montar o ambiente, descobrimos que `python -m truco` falha logo no início com `ValueError: Usecols do not match columns`.

Campo	Detalhe
Local	`truco/dados.py`, método `tratamento_inicial_df` (linha 12)
Causa	A base `dbtrucoimitacao_maos.csv` é separada por **tabulação**, mas o `pd.read_csv` usa o separador padrão, que é a vírgula. Conferimos no Git que o arquivo versionado também usa tabulação.
Impacto	Crítico: nenhuma partida pode ser jogada e a IA não carrega
Teste que comprova	`test_D01_jogo_carrega_base_de_casos_sem_adaptador`, que aparece como XFAIL
Contorno nos testes	Um adaptador de leitura no `conftest.py` detecta o separador. O código do jogo não foi alterado.

Atualização (11/11/2026): o grupo corrigiu o D01 em `truco/dados.py`, acrescentando `sep="\t"`. O alarme do `xfail_strict` acusou a correção: o teste passou a ser aprovado e por isso apareceu como falha. Retiramos o `xfail` e o caso CT44 agora tem dois testes aprovados, sem adaptador: a base carrega com o pandas original, e `python -m truco`, executado num processo separado, abre o jogo e mostra as cartas. O adaptador do `conftest.py` continua, mas só para os testes rodarem numa pasta temporária.

Testes de fumaça (test_00_ambiente.py)

Teste	O que verifica	Resultado
`test_todos_os_modulos_do_pacote_importam` (12 casos)	Cada módulo do pacote importa sem erro	Passou
`test_arquivos_de_dados_existem`	A base de casos e o modelo de registro estão presentes	Passou
`test_fabricas_de_carta_e_mao`	Cartas criadas a partir de texto	Passou
`test_baralho_fixo_entrega_cartas_na_ordem`	O baralho controlado entrega as cartas na ordem pedida	Passou

Teste	O que verifica	Resultado
`test_bot_com_mao_fixa_calcula_atributos`	O bot classifica a mão em Alta, Média e Baixa e calcula a qualidade	Passou
`test_simulacao_de_teclado`	O `input()` simulado responde e avisa quando as entradas acabam	Passou
`test_cbr_falsa_registra_chamadas`	A IA dublê devolve uma resposta por chamada	Passou
`test_ia_real_carrega_base_de_casos`	A IA real carrega mais de 20 mil casos, sem nenhum valor vazio	Passou
`test_partida_inicia_e_distribui_cartas`	A partida pede os dois nomes e dá 3 cartas a cada jogador	Passou
`test_partida_com_maos_fixas`	O cenário consegue fixar as mãos do humano e do bot	Passou
`test_D01_jogo_carrega_base_de_casos_sem_adaptador`	O jogo, rodado como o usuário roda, carrega a base de casos	XFAIL, comprova o D01

Como reproduzir

O jeito mais rápido é dar um duplo clique em `configurar_ambiente.bat`. Para fazer manualmente no Windows:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-test.txt
pytest -v
pytest --cov=truco --cov-report=html
```

O último comando gera o relatório de cobertura em `htmlcov/index.html`. A cobertura inicial ficou em 38% e vai crescer nas próximas etapas.

Etapa 2 – Requisitos e casos de teste

Resultado: levantamos 18 requisitos funcionais e 3 não funcionais, 9 cenários de uso e 44 casos de teste. Cada caso de teste aponta para um requisito e para uma categoria do enunciado. Esta seção é a entrega "documento de requisitos e cenários de uso".

Os requisitos vêm de três fontes: as regras do README, que seguem o site Jogatina, o comportamento descrito nas docstrings do código e os comentários do autor. Quando o README não define uma regra, como os pontos de um envido recusado, adotamos a regra implementada e a marcamos como "regra do sistema".

Requisitos funcionais

ID	Requisito	Fonte
RF01	O baralho é o espanhol, com 40 cartas: 1 a 7, 10, 11 e 12 dos naipes Espadas, Ouros, Copas e Bastos. Sem 8, 9 e curingas.	README
RF02	Cada jogador recebe 3 cartas, distribuídas de forma automática e aleatória.	README
RF03	A força das cartas, da menor para a maior, é 4, 5, 6, 7, 10, 11, 12, 1, 2, 3. As manilhas valem mais que todas, nesta ordem: 7 de Ouros, 7 de Espadas, 1 de Bastos e 1 de Espadas.	README e pontos.py
RF04	Na rodada vence a carta mais forte. Cartas de mesma força empatam.	README (definição de "Empatar")
RF05	A mão é disputada em melhor de 3 rodadas. Quem vence 2 rodadas ganha o valor da aposta, que é 1 ponto se ninguém pediu truco.	README
RF06	Quem vence uma rodada joga primeiro na rodada seguinte.	jogo.py
RF07	O truco eleva a mão a 2 pontos, o retruco a 3 e o vale 4 a 4. Quem recusa entrega a quem pediu o valor anterior (1, 2 ou 3). O mesmo jogador não pode aumentar a aposta duas vezes seguidas.	README e truco.py
RF08	Ao ir ao baralho, o jogador desiste da mão e o adversário ganha o valor atual da aposta.	README

ID	Requisito	Fonte
RF09	O envido só pode ser pedido na primeira rodada, com as 3 cartas na mão. Envido vale 2 pontos, Real Envido vale 5 e Falta Envido vale 12 menos os pontos do adversário. Recusar dá a quem pediu 1, 2 ou 5 pontos, respectivamente (regra do sistema).	README e envido.py
RF10	Os pontos de envido de uma mão são 20 mais a soma de duas cartas do mesmo naipe. Sem naipe repetido, valem os pontos da maior carta. As figuras 10, 11 e 12 valem 0.	Planilha de codificação e bot.py
RF11	No envido vence quem tem mais pontos. Em caso de empate vence o jogador 1, que é o mão (regra do sistema).	envido.py
RF12	Quem tem 3 cartas do mesmo naipe pode cantar Flor, que vale 3 pontos e bloqueia o envido. Se os dois têm flor, há Contraflor, que vale 6, ou Contraflor e o Resto.	README e flor.py
RF13	A partida termina quando um jogador chega a 12 pontos.	__main__.py
RF14	A cada jogada o jogador vê suas cartas e as opções válidas: [4] Truco, [5] Flor (só com flor), [6] Envido, [7] Real Envido, [8] Falta Envido (só com 3 cartas) e [9] Ir ao baralho.	jogador.py
RF15	Uma entrada inválida não encerra o jogo: o sistema mostra "Selecione um valor válido!" e pergunta de novo.	__main__.py
RF16	O bot decide qual carta jogar e como responder ao truco, ao envido e à flor usando CBR, a partir dos casos mais parecidos com a situação atual.	README e cbr.py
RF17	A IA usa a base `dbtrucoimitacao_maos.csv` e consulta os 100 vizinhos mais próximos.	README, cbr.py e comentários do autor
RF18	Ao fim de cada mão, o sistema registra as jogadas no arquivo `jogadas.csv`.	dados.py

Requisitos não funcionais

ID	Requisito	Fonte
RNF01	O jogo inicia com `python -m truco`, tendo pandas e scikit-learn instalados.	README
RNF02	Cada decisão da IA leva menos de 1 segundo, para que o jogo não pareça travado.	Critério do grupo
RNF03	Ao começar uma nova mão, todo o estado da mão anterior é zerado: cartas, rodadas, apostas, envido e flor.	Docstrings dos métodos `resetar`

Cenários de uso

ID	Cenário	Fluxo principal	Fluxos alternativos	Requisitos
UC01	Iniciar partida	O jogador informa o seu nome e o do bot, e cada um recebe 3 cartas.	Nome vazio.	RF01, RF02, RNF01
UC02	Jogar uma mão	O humano escolhe uma carta, o bot responde e quem vence a rodada joga primeiro. Após 2 vitórias o placar é atualizado e uma nova mão é dada.	O bot é o primeiro a jogar.	RF03 a RF06, RNF03
UC03	Humano pede truco	O humano escolhe [4] e o bot aceita, recusa ou aumenta.	Retruco e vale 4.	RF07, RF16
UC04	Bot pede truco	O bot pede truco e o humano responde [0] Recusar, [1] Aceitar ou [2] Aumentar.	Resposta inválida.	RF07, RF15
UC05	Envido	Um jogador pede [6], [7] ou [8] e o outro responde. O vencedor recebe os pontos.	Recusa, aumento para Real ou Falta Envido.	RF09 a RF11

ID	Cenário	Fluxo principal	Fluxos alternativos	Requisitos
UC06	Flor	O jogador com 3 cartas do mesmo naipe canta [5] e ganha 3 pontos.	Os dois têm flor e há Contraflor.	RF12
UC07	Ir ao baralho	O humano escolhe [9], o bot ganha o valor da aposta e uma nova mão é dada.	—	RF08
UC08	Entrada inválida	O humano digita uma letra ou um número fora das opções, o sistema avisa e pergunta de novo.	—	RF15
UC09	Fim de partida	Um jogador chega a 12 pontos, o sistema anuncia o vencedor e encerra.	—	RF13

Etapa 3 – Testes unitários

Resultado: implementamos os casos CT01 a CT30 em 7 arquivos. A suíte completa tem agora **288 testes aprovados e 27 falhas esperadas**, e roda em cerca de 5 segundos. A cobertura do pacote `truco` subiu de 38% para **75%**. Encontramos **10 defeitos novos** (D08 a D17), o que leva o total a 17.

Os casos CT31 a CT34 (entradas na tela principal e ir ao baralho) dependem da partida inteira rodando. Por isso foram para a Etapa 5, junto com os cenários de uso.

Arquivos e resultados

Arquivo	Casos de teste	Aprovados	Falhas esperadas	Defeitos comprovados
test_baralho_carta.py	CT01, CT02, CT03, CT05, CT06	90	2	D08
test_jogo.py	CT04, CT07, CT08, CT09	23	3	D02

Arquivo	Casos de teste	Aprovados	Falhas esperadas	Defeitos comprovados
test_envido.py	CT10 a CT17	41	7	D03, D04, D09, D10
test_truco.py	CT18 a CT23	18	8	D05, D06, D07, D09, D11, D12
test_flor.py	CT25 a CT27	23	2	D09, D14
test_jogador_bot.py	CT24 e métodos do Jogador e do Bot	40	2	D13, D15
test_interface.py	CT28 a CT30	32	2	D16, D17
test_00_ambiente.py (Etapa 1)	CT44	21	1	D01
Total		**288**	**27**	**17 defeitos**

Um teste parametrizado conta uma vez para cada combinação de parâmetros. Por isso o número de testes é maior que o de funções de teste.

Técnicas usadas

- **Parametrização (`@pytest.mark.parametrize`):** cobre as chamadas com parâmetros variados e as combinações de `if`s. Exemplos: os pontos das 40 cartas, uma por uma; os quatro tipos de confronto entre cartas (manilha ou comum de cada lado); as opções 0 a 3 do envido.
- **Oráculo independente:** a força esperada das cartas foi escrita no teste a partir do README, sem importar `pontos.py`. O teste `test_carta_vencedora_todos_os_pares_de_forca_diferente` confere sozinho os 1.560 confrontos possíveis do baralho.
- **Limites de laço:** o baralho é criado por `for n in range(1, 13)` com `if n < 8 or n >= 10`. Testamos os números 1, 7, 8, 9, 10, 12 e 13, ou seja, o início, o fim e as bordas do intervalo excluído. Nos laços `while` de validação testamos uma, duas e três respostas inválidas antes da válida, contando quantas vezes a pergunta foi feita.
- **Dublê de IA (`cbr_falsa`):** nos testes de truco, envido e flor, a resposta do bot é fixada pelo teste. Assim cada ramo dos `if`s é exercitado de propósito, e o teste também confere quais parâmetros o jogo passou para a IA. Foi assim que achamos o D10.

- **Teclado simulado (`entradas_usuario`):** troca o `input()` por uma lista de respostas e guarda cada pergunta exibida, para conferir o texto das perguntas.
- **Captura da tela (`capsys`):** compara as mensagens impressas, inclusive a moldura do placar, caractere por caractere.
- **Exceções (`pytest.raises`):** comprovam o `IndexError` ao tirar carta de um baralho vazio ou ao jogar um índice fora da mão.
- **Falha esperada estrita (`xfail` com `xfail_strict`):** cada defeito vira um teste que descreve o comportamento correto. Se alguém corrigir o jogo, o teste passa a aparecer como XPASS e avisa que a marcação deve ser removida.

Exemplo de teste parametrizado com um caso de defeito, em `test_envido.py`:

```
CASOS_ENVIDO = [
    pytest.param(MAO_33, 33, id="par-7-6-mesmo-naipe"),
    pytest.param(MAO_7, 7, id="naipes-diferentes"),
    pytest.param(("12 de COPAS", "7 de COPAS", "4 de OUROS"), 27,
                 id="figura-e-7-mesmo-naipe",
                 marks=[pytest.mark.defeito("D03"),
                        pytest.mark.xfail(reason="D03: par com figura vira
'maior carta'")]),
]

@pytest.mark.requisito("CT10")
@pytest.mark.parametrize("classe", [Jogador, Bot])
@pytest.mark.parametrize("mao, pontos", CASOS_ENVIDO)
def test_calcula_envido(nova_mao, classe, mao, pontos):
    assert classe("Teste").calcula_envido(nova_mao(*mao)) == pontos
```

Novos defeitos encontrados

ID	Defeito	Onde	Teste que comprova	Gravidade
D08	Com três cartas de mesma força (por exemplo, três 12), a mesma carta é classificada como Alta e Baixa. O bot quebra com `ValueError` ao receber essa mão.	carta.py, `classificar_carta`; bot.py, `calcular_qualidade_mao`	test_classificar_tres_cartas_de_mesma_forca e test_bot_recebe_tres_cartas_de_mesma_forca	Alta: derruba o jogo
D09	Uma resposta não numérica, como "sim", nas perguntas de truco, envido ou flor gera `ValueError` e encerra o jogo	truco.py, envido.py e flor.py: `int(input(...))`	test_entrada_nao_numerica_* (3 arquivos)	Alta: derruba o jogo

ID	Defeito	Onde	Teste que comprova	Gravidade
D10	Para decidir se está perdendo, o bot usa os próprios pontos de envido no lugar do placar do adversário	envido.py, chamadas a `avaliar_envido`	test_bot_empatado_no_placar_nao_esta_perdendo	Média: a IA decide com um dado errado
D11	O estado "vale_quatro" nunca é gravado. Depois do vale 4 um novo pedido é aceito e rebaixa a mão para 3 pontos.	truco.py, `pedir_vale_quatro`	test_depois_do_vale_4_nenhum_pedido_e_aceito	Média
D12	A mensagem "Jogador X aceitou o pedido" cita quem pediu, e não quem aceitou	truco.py	test_mensagem_cita_quem_aceitou	Baixa: só a mensagem
D13	`Jogador.resetar` não zera `pediu_flor`. Na mão seguinte o envido do bot fica bloqueado.	jogador.py	test_jogador_resetar_zera_pediu_flor	Média
D14	A Contraflor e o Resto vale só 3 pontos, menos que a Contraflor (6)	flor.py, `contraflor_resto`	test_contraflor_e_resto_vale_mais_que_a_contraflor	Média
D15	O bot nunca canta flor: `criar_mao` já marca `flor = True`, e por isso a condição de cantar nunca é verdadeira	bot.py, `jogar_carta`	test_bot_com_flor_canta_flor	Média
D16	A opção [5] Flor some a partir da segunda vez que as opções são exibidas	jogador.py, `mostrar_opcoes`	test_flor_continua_visivel_na_segunda_exibicao	Baixa
D17	Com 1 carta na mão, a opção [4] Truco não é exibida, embora a regra permita pedir truco na última rodada	jogador.py, `mostrar_opcoes`	test_truco_disponivel_na_ultima_rodada	Baixa

Atualização dos casos da Etapa 2: o CT22 confirmou o D12 e o CT24 confirmou o D13, que estavam como "defeito previsto". Todos os outros casos de CT01 a CT30 foram executados. Os que não estão ligados a um defeito passaram.

Cobertura por módulo

Módulo	Etapa 1	Etapa 3
baralho.py	78%	100%

Módulo	Etapa 1	Etapa 3
jogador.py	81%	100%
pontos.py	100%	100%
interface.py	28%	99%
jogo.py	36%	99%
carta.py	66%	97%
envido.py	29%	97%
truco.py	17%	94%
flor.py	19%	91%
bot.py	54%	89%
dados.py	40%	40%
cbr.py	39%	39%
__main__.py	25%	25%
Total	**38%**	**75%**

Os três módulos que ainda não subiram são a IA (`cbr.py` e `dados.py`) e o laço principal (`__main__.py`). Eles são o alvo das Etapas 4 e 5.

Etapa 4 – Testes da IA real e do registro de jogadas

Resultado: os casos CT35 a CT41 usam a **IA verdadeira**, com a base de 27.515 casos, em situações de jogo montadas pelos próprios métodos da classe `Dados`. São 62 testes aprovados e 17 falhas esperadas, em `test_cbr.py` e `test_dados.py`. Encontramos 7 defeitos novos (D18 a D24), quatro deles na lógica de decisão da IA.

Como a IA foi testada

- **Recuperação de casos:** um caso tirado da própria base precisa voltar como vizinho mais próximo, com distância 0. Testamos a primeira, uma do meio e a última linha da base. Também conferimos que a consulta devolve 100 vizinhos em ordem de distância.
- **Situações conhecidas:** a fixture `situacao` grava no registro da IA a mão do bot e a carta do humano, como o jogo faria. Com o registro inicial, a IA joga a carta mais fraca primeiro (o 4, de 1 ponto, numa mão com 1 de Espadas e 3). Uma mão forte nunca foge do truco. Com 6 ou mais pontos de envido o bot pede envido, e com até 4 não pede.

- **Escolha da carta isolada:** fixamos o valor de referência dos casos vencedores em 24 e conferimos que a IA joga a carta de pontuação mais próxima, em três mãos diferentes.
- **Propriedades:** as respostas ao truco sempre valem 0, 1 ou 2 e nunca diminuem quando a qualidade da mão sobe de 0 a 60. A mesma situação dá a mesma decisão em 5 consultas e em duas instâncias independentes da IA. Cada decisão leva cerca de 5 ms, bem abaixo do limite de 1 s do RNF02.
- **Rodada correta:** um espião em `DataFrame.__getitem__` registra qual coluna a IA consulta em cada rodada. Um cenário com a partida real confere se a situação gravada pelo jogo chega até a IA.
- **Registro de jogadas (CT41):** cada método da classe `Dados` é chamado com cartas conhecidas, e o teste confere a coluna e o valor gravados. O formato esperado vem da própria base: cada carta é gravada pelo código de pontos (52 para o 1 de Espadas, 24 para o 3).

Defeitos encontrados na IA e no registro

ID	Defeito	Teste que comprova	Gravidade
D18	O filtro de "jogadas vencidas" tem os parênteses errados. As 1.996 mãos em que o bot venceu a 1ª e a 2ª rodada ficam de fora da referência, cerca de 15% das vitórias.	test_filtro_considera_todas_as_maos_vencidas	Média
D19	Quando nenhum dos 100 vizinhos é uma mão vencida, `jogar_carta` lança `IndexError`. Acontece, por exemplo, com duas manilhas na mão ou na 2ª rodada depois de perder a 1ª.	test_primeira_rodada_devolve_indice_valido[forte] e test_segunda_rodada_depois_de_perder_a_primeira	Alta: derruba o jogo
D20	As decisões de truco e envido comparam os rótulos (2 > 1), e não as contagens. Com qualquer mão real o bot aumenta o truco, e a resposta ao envido é a mesma com 0 ou com 33 pontos.	test_mao_fraca_nao_aumenta_a_aposta e test_resposta_depende_dos_pontos_do_bot	Alta: a IA não usa a mão
D21	As rodadas estão invertidas: a rodada 1 consulta `terceiraCartaRobo` e a rodada 3 consulta `primeiraCartaRobo`	test_rodadas_1_e_3_consultam_a_propria_carta	Média

ID	Defeito	Teste que comprova	Gravidade
D22	A IA cria a sua própria instância de `Dados`, e o jogo grava a situação em outra. A IA sempre consulta o registro zerado e nunca "vê" a partida.	test_ia_recebe_a_situacao_da_partida	Alta: anula o CBR
D23	O registro grava em colunas erradas: as cartas do robô vão sempre para `terceiraCartaRobo`, e os campos `SegundaCartaHumano`, `qualidadeMaoBot`, `quemEnvido`, `quemRealEnvido` e `quemFaltaEnvido` não existem na base	5 testes em test_dados.py	Média
D24	As cartas são gravadas pelo número (1), e não pelo código da base (52). Assim, um 1 de Espadas jogado é registrado como se fosse um 4.	test_carta_gravada_com_o_codigo_da_base	Média

Também confirmamos o D15 do lado da IA: o bot chama `cbr.flor()`, mas a classe `Cbr` não tem esse método.

Observação importante: o D22 esconde o D19 durante uma partida. Como a IA sempre recebe o registro zerado, ela nunca encontra a situação que a faz quebrar. Se o D22 for corrigido sozinho, o D19 passa a derrubar o jogo. Por isso os testes do D19 montam a situação diretamente no registro da IA.

Etapa 5 – Entradas do usuário e cenários de partida

Resultado: os casos CT31 a CT34, CT42 e CT43 rodam o jogo inteiro (`truco/__main__.py`) com teclado simulado, mãos fixas e a IA verdadeira. Somando os cenários extras de truco e envido e o CT17, são 22 testes aprovados e 10 falhas esperadas. Com as mãos fixas e o registro inicial, a IA é determinística, então cada cenário segue sempre o mesmo caminho. Encontramos 2 defeitos novos (D25 e D26), e D05 e D12 também apareceram dentro de uma partida real.

Como uma partida é executada no teste

A fixture `executar_partida` executa o código do `__main__.py` sem alterá-lo. Ela recebe a lista do que o usuário digita e as mãos de cada rodada, e devolve três coisas: tudo o que apareceu na tela, cada pergunta feita e as variáveis do jogo no fim (jogadores, pontos, aposta e a IA). Quando as entradas acabam, a partida para com `EntradasEsgotadas`. Isso mostra que o jogo continuou funcionando até pedir a próxima jogada.

Exemplo do cenário CT43, em que o bot chega a 12 pontos depois que Renato vai ao baralho 12 vezes:

```
@pytest.fixture
def partida(self, executar_partida):
    return executar_partida("Renato", "Lucas", *["9"] * 12,
                            maos=[(HUMANO_FORTE, BOT_FRACO)] * 13)

def test_partida_termina_aos_12_pontos(self, partida):
    assert partida.erro is None          # o laço principal terminou sozinho
    assert pontos(partida) == (0, 12)
    assert "Lucas ganhou o jogo" in partida.saida
```

Cenários executados

Caso	Entradas	O que conferimos	Resultado
CT31	"a", "1.5" ou vazio na escolha da carta	Aviso e nova pergunta	Falhou: D09 (`ValueError` encerra o jogo)
CT32	"-1", "5" (sem flor), "12" e as três seguidas	"Selecione um valor válido!" uma vez para cada uma, e depois a carta é jogada	Passou
CT32	"3" com 3 cartas e "2" com 2 cartas	Aviso e nova pergunta	Falhou: D25 (`IndexError`)
CT33	"9" no início e depois de vencer a 1ª rodada	O bot ganha 1 ponto, a tela diz "Jogador Renato fugiu!", uma mão nova é dada e a mão é gravada em `jogadas.csv`	Passou
CT34	Nomes comuns, nomes vazios, nome com espaço e número	Os nomes aparecem nas mensagens e o jogo segue normalmente	Passou

Caso	Entradas	O que conferimos	Resultado
CT42	Renato joga o 1 de Espadas e o 1 de Bastos, e aceita o truco do bot	Duas vitórias, placar, nova mão e registro em `jogadas.csv`	Passou. A pontuação dá 1 ponto, quando deveria dar 2 (D05)
CT43	"9" doze vezes	A partida termina aos 12 pontos com "Lucas ganhou o jogo". O laço roda 12 vezes e nada é perguntado depois do fim.	Passou
CT17	Na vez do bot, a IA decide pedir envido	O envido acontece	Falhou: D04 (`TypeError`)
UC03	Renato pede truco e o bot aumenta para retruco	A pergunta informa "3 pontos" e a aposta vai a 3	Passou
UC04	Renato recusa o truco pedido pelo bot	O bot ganha 1 ponto e uma nova mão começa	Passou. A tela atribui a fuga ao bot (D26) e a pergunta começa com "2, você aceita" (D12)
UC05	O bot, com 33 pontos de envido, pede envido no início e Renato aceita	O bot ganha 2 pontos e a partida continua	Passou. A pergunta diz "Jogador 2, você aceita" (D12)

CT17 agora executa o turno real do bot

Antes, o teste copiava para dentro dele a chamada defeituosa da linha 128 do `__main__.py` . Se o jogo fosse corrigido, o teste continuaria falhando, porque a cópia ficaria errada. Agora o teste fixa apenas a decisão do bot ("pedir envido" na primeira jogada) e deixa o `turno_do_bot` verdadeiro rodar. Quando o D04 for corrigido, o teste passa sozinho, e o `xfail_strict` avisa que a marcação pode sair.

Reforço do CT17 (revisão do grupo): logo após a chamada defeituosa, as linhas 129 a 136 chamam de novo o `controlador_envido` , mas com `quem_pediu = 1` , como se o humano tivesse pedido. Se alguém apagasse só a linha 128, o envido aconteceria "em nome do humano": o próprio bot responderia e o humano nem seria perguntado. A versão antiga do teste aprovaria isso. Agora dois espiões acompanham a partida, um no `controlador_envido` e outro no `avaliar_envido` , e o teste confere três coisas: o turno

não quebra, todo pedido é registrado com o **bot como autor** (`quem_pediu = 2`), e o **humano recebe a pergunta** do envido, sem que a IA seja chamada para responder ao próprio pedido.

Comprovamos o reforço com dois mutantes do jogo, numa cópia do projeto:

Alteração no `__main__.py`	Resultado do CT17
Sem alteração	XFAIL: `TypeError` (D04)
Só a linha 128 apagada	XFAIL: o teste mostra `pedidos registrados: [(6, 1), ...]`, ou seja, o pedido saiu em nome do humano
Linha 128 apagada e `quem_pediu = 2` nas linhas 130, 133 e 136	XPASS(strict): o teste passa e avisa que o `xfail` pode ser retirado

Novos defeitos

ID	Defeito	Onde	Gravidade
D25	O teste `carta_escolhida <= len(mao)` aceita um índice igual ao número de cartas. Digitar "3" com 3 cartas dá `IndexError` e encerra o jogo.	__main__.py, linha 51	Alta: derruba o jogo
D26	Quando o humano recusa o truco do bot, a tela mostra "Jogador Lucas fugiu!". O `__main__` interpreta o -1 do turno do bot como fuga do bot.	__main__.py, linhas 197 a 205	Baixa: só a mensagem

Etapa 6 – Resultado final

Resultado: a suíte completa tem **427 testes: 374 aprovados, 53 falhas esperadas e nenhuma falha inesperada**. Ela roda em cerca de 20 segundos e cobre **91%** das linhas do pacote `truco`. Os 44 casos de teste foram executados (veja a coluna Situação na Etapa 2). Registramos 26 defeitos: 1 corrigido pelo grupo e 25 em aberto, dos quais 8 são de gravidade alta.

Validamos a entrega extraindo o ZIP numa cópia limpa do projeto e rodando `pytest` do zero. O resultado foi o mesmo: 374 aprovados e 53 falhas esperadas.

Resultado por arquivo

Arquivo	Casos de teste	Aprovados	Falhas esperadas
test_00_ambiente.py	CT44 e fumaça	23	0
test_baralho_carta.py	CT01 a CT06	90	2
test_jogo.py	CT04 a CT09	23	3
test_envido.py	CT10 a CT17	41	8
test_truco.py	CT18 a CT23	18	8
test_jogador_bot.py	CT24	40	2
test_flor.py	CT25 a CT27	23	2
test_interface.py	CT28 a CT30	32	2
test_entradas_usuario.py	CT31 a CT34	11	5
test_cbr.py	CT35 a CT40	52	11
test_dados.py	CT41	10	6
test_cenarios.py	CT42, CT43, UC03 a UC05	11	4
Total	**44 casos**	**374**	**53**

Resultado por categoria (marcadores)

Comando	Aprovados	Falhas esperadas
`pytest -m ia`	53	11
`pytest -m entrada`	25	10
`pytest -m cenario`	24	11
`pytest -m requisito`	288	50
`pytest -m defeito`	0	53
`pytest -m "not lento"` (rápido, cerca de 2 s)	286	26

Cobertura de linhas por módulo

Módulo	Etapa 1	Etapa 3	Final
baralho.py	78%	100%	100%
dados.py	40%	40%	100%
jogador.py	81%	100%	100%
pontos.py	100%	100%	100%
interface.py	28%	99%	99%
jogo.py	36%	99%	99%
carta.py	66%	97%	97%
envido.py	29%	97%	97%
truco.py	17%	94%	94%
bot.py	54%	89%	91%
flor.py	19%	91%	91%
cbr.py	39%	39%	82%
__main__.py	25%	25%	73%
Total	**38%**	**75%**	**91%**

As linhas que faltam no `__main__.py` e no `cbr.py` são, em boa parte, trechos que o próprio jogo não consegue alcançar. Exemplos: o bot cantando flor, impedido pelo D15; o ramo depois do `TypeError` do D04; e o método `carregar_dataset`, que nunca é chamado.

Registro consolidado de defeitos

ID	Defeito	Gravidade	Casos de teste	Situação
D01	A base separada por tabulação era lida com vírgula, e o jogo não iniciava	Crítica	CT44	Corrigido pelo grupo
D02	Cartas de mesma força não empatam	Média	CT07	Aberto

ID	Defeito	Gravidade	Casos de teste	Situação
D03	O envido de um par com figura é calculado errado	Média	CT10	Aberto
D04	O bot quebra ao pedir envido na sua vez, e o pedido seguinte é registrado como do humano (`TypeError`)	Alta	CT17	Aberto
D05	Truco aceito não muda o valor da mão	Alta	CT18, CT22, CT42	Aberto
D06	O vale 4 nunca acontece em pedidos separados	Média	CT19	Aberto
D07	`retornar_quem_fugiu` retorna o próprio método	Baixa	CT23	Aberto
D08	Três cartas de mesma força quebram o bot	Alta	CT06	Aberto
D09	Texto em vez de número derruba o jogo	Alta	CT11, CT22, CT27, CT31	Aberto
D10	O bot avalia o envido com os próprios pontos de envido, e não com o placar	Média	CT12	Aberto
D11	Depois do vale 4, um novo pedido rebaixa a mão para 3	Média	CT21	Aberto
D12	As mensagens de truco e envido citam o jogador errado	Baixa	CT11, CT22, UC04, UC05	Aberto
D13	`Jogador.resetar` não zera `pediu_flor`	Média	CT24	Aberto
D14	A Contraflor e o Resto vale só 3 pontos	Média	CT26	Aberto
D15	O bot nunca canta flor, e a classe `Cbr` não tem `flor()`	Média	test_cbr.py e test_jogador_bot.py	Aberto
D16	A opção [5] Flor some na segunda exibição	Baixa	CT28	Aberto

ID	Defeito	Gravidade	Casos de teste	Situação
D17	A opção [4] Truco não aparece com 1 carta	Baixa	CT28	Aberto
D18	O filtro de jogadas vencidas ignora cerca de 15% das vitórias	Média	CT36	Aberto
D19	A IA quebra quando nenhum vizinho é caso vencedor	Alta	CT36	Aberto (escondido pelo D22)
D20	As decisões de truco e envido não dependem da mão	Alta	CT37, CT38	Aberto
D21	As rodadas 1 e 3 consultam colunas invertidas	Média	CT40	Aberto
D22	A IA nunca recebe a situação da partida	Alta	CT40	Aberto
D23	O registro de jogadas grava em colunas erradas ou inexistentes	Média	CT41	Aberto
D24	As cartas são gravadas pelo número, e não pelo código da base	Média	CT41	Aberto
D25	Um índice igual ao número de cartas derruba o jogo	Alta	CT32	Aberto
D26	A fuga no truco é atribuída ao jogador errado	Baixa	UC04	Aberto

Resumo: 1 crítico (corrigido), 8 de gravidade alta, 12 médios e 5 baixos. Os defeitos altos se dividem em dois grupos. Cinco derrubam o jogo: D04, D08, D09, D19 e D25. Três fazem as regras ou a IA não funcionarem como prometido: D05, D20 e D22.

Etapa 7 – Entrega

Resultado: o arquivo `RenatoLucastestesoftware.zip` está pronto, na pasta do projeto, com os 12 arquivos de teste, a configuração e as instruções. Falta gravar o vídeo `RenatoLucastestesoftware.mp4` e enviar os dois arquivos e este documento ao professor até 23/11/2026, às 14h30.

Conteúdo do ZIP

Arquivo	Para que serve
`LEIA-ME.txt`	Como instalar e rodar, e o resultado esperado (374 aprovados e 53 falhas esperadas)
`configurar_ambiente.bat`	Cria o `.venv`, instala as dependências e roda os testes
`pytest.ini`	Configuração e marcadores
`requirements.txt` e `requirements-test.txt`	Dependências do jogo e dos testes
`tests/conftest.py`	Fixtures compartilhadas
`tests/test_*.py` (12 arquivos)	Os testes
`tests/README.md`	Comandos, como ler o resultado e o mapa arquivo → casos de teste

O ZIP não inclui o código do jogo nem a base de 14 MB. Basta extrair o conteúdo na raiz do projeto, ao lado da pasta `truco/`.

Checklist de entrega

- [x] ~~Documento de requisitos e cenários de uso: a Etapa 2 deste relatório (exportar em PDF)~~
- [x] ~~Scripts de teste em `RenatoLucastestesoftware.zip`~~
- [ ] Vídeo `RenatoLucastestesoftware.mp4`, de 5 a 15 minutos
- [ ] Enviar o ZIP e o MP4 por e-mail ao professor até 23/11/2026, às 14h30

Roteiro do vídeo (cerca de 10 minutos)

O enunciado pede para mostrar o código, executar os testes e explicar os resultados, sem slides. Sugestão de ordem, com o terminal e o editor lado a lado:

Tempo	O que mostrar	Comando ou arquivo
0:00 a 1:00	Apresentação: o jogo, a IA com CBR e a decisão de testar sem corrigir o jogo, usando `xfail` para registrar cada defeito	README.md do jogo

Tempo	O que mostrar	Comando ou arquivo
1:00 a 2:00	Ambiente: `.venv`, `pytest.ini`, marcadores e fixtures principais (teclado simulado, IA dublê e partida completa)	`pytest.ini` e `tests/conftest.py`
2:00 a 3:00	Suíte inteira rodando e o resumo final	`pytest -v`
3:00 a 4:00	Requisitos e ifs: os 1.560 confrontos de cartas e o envido parametrizado	`test_jogo.py` e `test_envido.py`, com `pytest -m requisito`
4:00 a 5:00	Entradas do usuário: opção inválida repetindo a pergunta, e texto derrubando o jogo (D09)	`pytest -m entrada -v`
5:00 a 7:00	IA: recuperação com distância 0, decisões em situações conhecidas e os defeitos D20 e D22	`test_cbr.py`, com `pytest -m ia -v`
7:00 a 8:30	Cenários: mão completa com truco (D05 na partida real) e fim aos 12 pontos	`test_cenarios.py`, com `pytest -m cenario -v`
8:30 a 9:30	Defeitos e o alarme do `xfail_strict` no D01 corrigido	`pytest -m defeito -rx`
9:30 a 10:30	Cobertura: de 38% para 91% e o relatório HTML	`pytest --cov=truco --cov-report=html` e depois `htmlcov/index.html`

Dica: rode `pytest` uma vez antes de gravar, para o pandas e a base já estarem no cache do disco e a execução no vídeo ficar rápida.

Casos de teste

O resultado esperado segue o requisito, e não o que o código faz hoje. Na coluna Situação, "Defeito confirmado" indica que já executamos o caso à mão e o jogo falhou. "Defeito previsto" indica uma suspeita vinda da leitura do código, que será confirmada nas próximas etapas.

As categorias usam estas siglas: REQ (requisitos e cenários), PAR (chamadas com parâmetros variados), ENT (entrada do usuário), RET (retorno de função), IA, EXC (exceções), MSG (mensagens), LAÇO e IF.

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT01	RF01	REQ, LAÇO	Criar um baralho	40 cartas, 10 por naipe, sem 8 nem 9 e sem repetição	test_baralho_carta.py	Passou
CT02	RF02	PAR, LAÇO	Dar a mão ao humano e ao bot	3 cartas para cada um e 34 cartas no baralho	test_baralho_carta.py	Passou
CT03	RF02	RET	Embaralhar com sementes diferentes	As mesmas 40 cartas em outra ordem	test_baralho_carta.py	Passou
CT04	RF03	IF, PAR	Todos os pares entre manilha e manilha, manilha e comum, comum e manilha, comum e comum	Vence a carta mais forte pela tabela do RF03	test_jogo.py	Passou (1.560 confrontos)
CT05	RF03	RET	`retornar_pontos_carta` para as 40 cartas	Manilhas valem 52, 50, 42 e 40. As comuns vão de 24 (o 3) a 1 (o 4).	test_baralho_carta.py	Passou
CT06	RF03	IF, RET	`classificar_carta` com mãos variadas, com e sem manilha	Classificação Alta, Média e Baixa correta	test_baralho_carta.py	Passou, exceto com três cartas de mesma força (D08)
CT07	RF04	IF	3 de Copas contra 3 de Ouros	Empate	test_jogo.py	Falhou: D02
CT08	RF05	RET	`adicionar_rodada` com o vencedor 1, o vencedor 2 e um ganhador inválido	Retorna 1 ou 2 e soma uma rodada ao vencedor. Com ganhador inválido retorna "Erro".	test_jogo.py	Passou
CT09	RF06	IF	`quem_joga_primeiro` após vitória do humano e após vitória do bot	O vencedor fica com `primeiro = True`	test_jogo.py	Passou
CT10	RF10	PAR, RET	`calcula_envido` com 7+6 de Copas, naipes todos diferentes, 12+7 de Copas e 10+11 de Copas	33, 7, 27 e 20	test_envido.py	Falhou com figura no par: D03
CT11	RF09	ENT, LAÇO	O humano responde ao envido do bot com 0, 1, 2 e 3, e com "9" seguido de "1"	Cada opção é aceita. A opção inválida faz repetir a pergunta.	test_envido.py	Passou com números. Falhou com texto (D09) e no texto da pergunta (D12)
CT12	RF09	REQ	Envido recusado e envido aceito	Recusado: quem pediu ganha 1. Aceito: o vencedor ganha 2.	test_envido.py	Passou. A IA recebe um dado errado (D10)
CT13	RF09	REQ	Real Envido aceito e recusado	O vencedor ganha 5. Na recusa, quem pediu ganha 2.	test_envido.py	Passou

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT14	RF09	REQ	Falta Envido com o adversário em 0 e em 8 pontos	Vale 12 e 4 pontos. A recusa dá 5 a quem pediu.	test_envido.py	Passou
CT15	RF11	IF	Pontos de envido: J1 maior, J2 maior e empate	Vence J1, J2 e J1, respectivamente	test_envido.py	Passou
CT16	RF09	IF, RET	Segundo pedido de envido na mesma mão e pedido de jogador bloqueado	`controlador_envido` retorna None e não muda os pontos	test_envido.py	Passou
CT17	RF09	REQ, EXC	O bot escolhe envido na vez dele (6, 7 ou 8)	O envido acontece normalmente	test_envido.py	Falhou no turno real do bot: D04
CT18	RF07	REQ	Truco aceito	A mão passa a valer 2	test_truco.py	Falhou: D05 (também numa partida real)
CT19	RF07	REQ, IF	Truco, retruco e vale 4, todos aceitos	A mão vale 3 e depois 4	test_truco.py	Passou numa resposta só. Falhou em pedidos separados: D06
CT20	RF07	REQ	Recusas de truco, retruco e vale 4	Quem pediu ganha 1, 2 e 3 pontos	test_truco.py	Passou. Mensagem de fuga errada (D26)
CT21	RF07	IF, RET	O mesmo jogador pede truco duas vezes seguidas, ou pede após o vale 4	`controlador_truco` retorna None	test_truco.py	Passou, exceto após o vale 4: D11
CT22	RF07	ENT, LAÇO, MSG	O humano responde ao truco do bot com 0, 1, 2 e com uma opção inválida	A pergunta mostra o valor da mão e repete na opção inválida. A mensagem cita quem aceitou.	test_truco.py	Passou nas opções inválidas. Falhou: D05, D09 e D12
CT23	RF07	RET	`retornar_valor_aposta` e `retornar_quem_fugiu`	Retornam 1 e o número do jogador	test_truco.py	Falhou: D07
CT24	RNF03	RET	`resetar` de Truco, Envido, Flor, Jogador e Bot	Todos os atributos voltam ao valor inicial	test_jogador_bot.py	Falhou: D13
CT25	RF12	PAR, IF	`checa_flor` com 3 cartas do mesmo naipe e com 2+1	True e False	test_flor.py	Passou
CT26	RF12	REQ	Só o humano tem flor, só o bot tem e os dois têm (Contraflor aceita ou recusada)	+3 para quem tem flor, +6 para o vencedor da Contraflor	test_flor.py	Passou, exceto a Contraflor e o Resto: D14

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT27	RF12	ENT, LAÇO	`decisao_jogador` com "5" e depois "1"	Repete a pergunta e depois retorna True	test_flor.py	Passou com números. Falhou com texto: D09
CT28	RF14	MSG, IF	`mostrar_opcoes` com 3, 2 e 1 cartas, com e sem flor	Opções exibidas de acordo com o RF14	test_interface.py	Falhou: D16 e D17
CT29	RF14	MSG	`border_msg`, placar, vencedor do envido e vencedor da flor	Texto e moldura corretos	test_interface.py	Passou
CT30	RF14	MSG, IF	`desenhar_cartas` para os 4 naipes	Símbolos ♠ ♦ ♥ ♣ corretos	test_interface.py	Passou
CT31	RF15	ENT, EXC	Na escolha de carta, o humano digita "a"	Aviso e nova pergunta	test_entradas_usuario.py	Falhou: D09
CT32	RF15	ENT, EXC	Com 3 cartas na mão, o humano digita "3"	Aviso e nova pergunta	test_entradas_usuario.py	Falhou com índice igual ao número de cartas: D25
CT33	RF08	REQ, ENT	O humano escolhe [9] Ir ao baralho	O bot ganha 1 ponto e uma nova mão é dada	test_entradas_usuario.py	Passou
CT34	UC01	ENT, MSG	Nomes "Renato" e "Lucas", e nome vazio	Os nomes aparecem nas mensagens e o nome vazio não quebra o jogo	test_entradas_usuario.py	Passou
CT35	RF17	IA	Carregar a base de casos	27.515 casos, sem vazios, com as mesmas colunas do registro	test_cbr.py	Passou
CT36	RF16	IA, RET, PAR	`jogar_carta` com mãos variadas nas rodadas 1, 2 e 3	Retorna um índice válido da mão	test_cbr.py	Falhou: D18 e D19
CT37	RF16	IA, RET	`truco` com qualidade de mão muito baixa, média e alta	Retorna 0, 1 ou 2. Com mão fraca foge (0) e com mão forte não foge.	test_cbr.py	Passou nas respostas válidas. Falhou: D20
CT38	RF16	IA, RET	`envido` para cada tipo e para o pedido do próprio bot	Tipo 6 retorna de 0 a 3, tipos 7 e 8 retornam 0 ou 1, o pedido do bot retorna 6, 7 ou 8	test_cbr.py	Passou nas respostas válidas. Falhou: D20
CT39	RF16, RNF02	IA	A mesma situação consultada duas vezes	A mesma decisão, em menos de 1 s	test_cbr.py	Passou (cerca de 5 ms por decisão)
CT40	RF16	IA, IF	Rodada 1 em `jogar_carta`	Consulta a coluna da primeira carta do robô	test_cbr.py	Falhou: D21 e D22
CT41	RF18	REQ	`Dados` grava as cartas da segunda e da terceira rodada	Cada carta é gravada na coluna certa	test_dados.py	Falhou: D23 e D24

ID	Req.	Categoria	Entrada / situação	Resultado esperado	Arquivo de teste	Situação
CT42	UC02	REQ, cenário	Partida com mãos fixas em que o humano vence 2 rodadas	O humano ganha 1 ponto, o placar aparece e uma nova mão é dada	test_cenarios.py	Passou, exceto a pontuação com truco: D05
CT43	UC09	REQ, cenário, LAÇO	Um jogador chega a 12 pontos	A mensagem "... ganhou o jogo" aparece e o laço principal termina	test_cenarios.py	Passou
CT44	RNF01	REQ, EXC	`python -m truco`	O jogo inicia	test_00_ambiente.py	Passou (D01 corrigido)

Cobertura das categorias do enunciado

Categoria do enunciado	Casos de teste
Requisitos e cenários de uso (obrigatório)	CT01, CT12 a CT14, CT17 a CT20, CT26, CT33, CT41 a CT44
Chamadas com parâmetros variados (obrigatório)	CT02, CT04, CT10, CT25, CT36
Entradas do usuário (obrigatório)	CT11, CT22, CT27, CT31 a CT34
Retornos de função (obrigatório)	CT03, CT05, CT06, CT08, CT10, CT16, CT21, CT23, CT24, CT36 a CT38
IA do jogo (obrigatório)	CT35 a CT40
Exceções	CT17, CT31, CT32, CT44
Mensagens de erro e de tela	CT22, CT28 a CT30, CT34
Início e fim de laços	CT01, CT02, CT11, CT22, CT27, CT43
Combinações de `if`s	CT04, CT06, CT07, CT09, CT15, CT16, CT19, CT21, CT25, CT28, CT30, CT40

Defeitos registrados até aqui

ID	Defeito	Caso de teste	Situação
D01	A base de casos é separada por tabulação e o jogo a lê com vírgula, por isso não inicia	CT44	Corrigido pelo grupo

ID	Defeito	Caso de teste	Situação
D02	Cartas de mesma força não empatam: vence sempre a carta do jogador 1	CT07	Confirmado
D03	O envido de duas cartas do mesmo naipe com figura é calculado errado	CT10	Confirmado
D04	O bot pede envido com os argumentos errados (TypeError)	CT17	Confirmado
D05	Truco aceito não muda o valor da mão, que continua em 1	CT18	Confirmado
D06	Depois do retruco, um novo pedido chama retruco de novo, e o vale 4 nunca acontece	CT19	Confirmado
D07	`retornar_quem_fugiu` retorna o próprio método, e não um valor	CT23	Confirmado